Migrant Archive Agent

Cero: An access layer for migrant narratives

Sebastián López Largo

Migrant Archive Agent

Cero: An access layer for migrant narratives

Ironhack - AI Engineering Bootcamp Final Project

Abstract

Migrant Archive Agent is an AI project that works with Plataforma Cero's migrant narratives documented from events in Barcelona. The project answers a practical problem: important information is often inside long videos, weak captions, and formats that are hard to search. Cero, the user-facing agent, helps people ask questions, find specific moments, and receive answers connected to the original video context.

The project combines transcription, semantic search, a vector database, an agent with tools and memory, a FastAPI backend, voice input, and an embeddable web widget. The goal is to make migrant knowledge searchable, reusable, and easy to connect with.

1. Context and Purpose

The project was developed for Plataforma Cero, a Barcelona-based association that creates social and cultural projects for migrant communities. The goal is to make migrant stories easy to find and use. This helps people connect with those narratives, write their own stories, and find tools for social participation in Barcelona.

The source material is public, but it still needs verification from the organization before any wider or production use. For that reason, the first version was built and tested locally. This local-first approach supports that verification step before moving toward a more open deployment.

The technical challenge is clear: long videos bury useful knowledge. A person may need one idea, one quote, or one moment, but finding it manually is slow. This is harder when the content includes Spanish and Catalan, when captions are weak, or when the user needs a direct answer instead of a full video.

2. System Overview

The system follows a step-by-step pipeline:

1. **Ingestion:** YouTube videos are collected and converted into structured transcript JSON files.
2. **Processing:** Transcripts are divided into chunks with timestamps and metadata.
3. **Embeddings:** Each chunk is converted into a Gemini embedding.
4. **Vector storage:** ChromaDB stores the chunks and allows semantic search.
5. **Validation:** Small scripts test retrieval before the agent layer is used.
6. **Agent:** Cero uses LangChain tool calling, memory, and three tools to answer questions.
7. **Evaluation:** After the agent is created, RAGAS evaluates the quality of the answers and the retrieved context.
8. **API:** FastAPI exposes the agent to the frontend.
9. **Widget:** A Vite and TypeScript widget gives users a simple interface with text and voice input.
10. **Observability:** LangSmith traces the system once the agent and product surface exist.

This structure keeps the project understandable. Each part has one main responsibility, and each part can be tested before moving to the next one.

3. Main Technical Decisions

Area	Choice	Reason
Transcription	faster-whisper 1.2.1 on Colab GPU	Speed and transcript control
Vector store	ChromaDB	Local and inspectable
Embeddings	gemini-embedding-2	Multilingual, 3072 dimensions
Chunking	1000 tokens, 200 overlap	Preserves ideas in speech
Agent	LangChain native tool calling	Tool-routing instead of generic search
Evaluation	RAGAS	Quality scoring after generation
API	FastAPI	Simple backend interface
Frontend	Vite and TypeScript	Embeddable typed widget
Observability	LangSmith	Agent call tracing

Transcription workflow

YouTube captions were useful as a quick reference, but they were not enough as the main source. Spanish and Catalan content can be difficult to process accurately with platform captions, and the transcript format needs to be stable enough for search, timestamps, and later processing.

The transcription step used faster-whisper 1.2.1 on a Colab GPU because the local CPU option was slower and offered less control over the transcript output. After transcription, the other steps were run locally when possible. This made the workflow practical: use GPU when it gives a clear benefit, and keep the rest of the system local and easy to inspect.

RAG and vector search

The project required a RAG system, a vector database, agent tools, memory, and evaluation. A RAG system also fit the project's goal: users need specific answers backed by sources, not general summaries.

ChromaDB was selected because it works locally, requires no external vector database account, and is enough for the project scale. Gemini embeddings (`gemini-embedding-2`, 3072 dimensions) were used to represent transcript chunks as vectors. The system uses 1000-token chunks with 200-

token overlap, which helps preserve complete ideas in conversational Spanish.

The same model, `gemini-embedding-2`, is used to create transcript vectors and to perform similarity search against them later through the agent. This is important because semantic search works correctly when stored chunks and user questions are represented in the same embedding space.

Retrieval validation script

Before building the full agent, semantic search needed to be tested directly. For that reason, RAG Test was wrapped in a script that can be called from the terminal. This made it possible to check if retrieval worked before adding memory, tool calling, API routes, or frontend logic.

If RAG Test returned weak results, the problem was probably in chunking, embeddings, or the vector store. If RAG Test worked but the agent failed, the problem was probably in the agent layer.

Agent tools and memory

Cero uses three tools: `list_videos`, `get_video_info`, and `search_transcripts`. This tool structure helps the agent choose the right action instead of treating every question as a generic search. For broad or vague questions, the agent can call `list_videos` or `get_video_info` first to narrow the scope; once it has a clearer target, it calls `search_transcripts` with better context. This is tool-routing and disambiguation, not a separate query-rewriting model. The agent uses Gemini 2.5 Flash as its language model.

Memory was added so the conversation can continue across follow-up questions. For example, after asking about one speaker or one video, the next question can refer to that context without repeating everything.

API, widget, and voice input

The browser does not run the AI model directly. The frontend calls a FastAPI backend, and the backend runs the agent. This keeps the frontend lighter and keeps API keys outside the browser.

Voice input uses the Groq Whisper API. Audio interaction was a project requirement, and it also supports the accessibility goal: not every user types.

Observability

LangSmith was added after the agent and product surface existed. It traces agent calls for debugging and observability, while answer quality is evaluated separately with RAGAS.

4. Final Product

Cero is the final user-facing layer of the project. It allows users to ask questions about the indexed videos and receive grounded answers with source context. The widget can be embedded into a website, which makes the system easier to test inside a realistic Plataforma Cero page.

Cero provides:

- semantic search inside long videos,
- answers based on transcript context,
- source-backed references with timestamps,
- catalog tools for browsing videos,
- conversation memory,
- text and voice input,
- 6-language i18n (EN/ES/CA/FR/PT/DE).

The result is both an access layer and a chatbot. It turns long migrant narrative videos into searchable knowledge with source references.

5. Testing and TDD Process

The project followed a TDD workflow: red, green, and refactor. Tests were written to describe expected behavior, then the implementation was adjusted until those tests passed. The suite uses pytest and covers the pipeline from ingestion to evaluation.

Tests are organized in three layers: unit tests with fake providers, integration tests with an in-memory ChromaDB instance, and end-to-end tests that can use the real Gemini API when the key is available. This helped check behavior by parts instead of waiting until the end.

The testing and review process was influenced by TDD practice and by open-source review workflows such as Gentleman Guardian Angel (https://github.com/Gentleman-Programming/gentleman-guardian-angel), created by Alan Buscaglia. This helped shape the habit of checking behavior, standards, and regressions during development, not only at the end.

Main project paths:

```
migrant-archive/
├── backend/
│   ├── core/
│   │   ├── ingestion.py
│   │   ├── processor.py
│   │   └── vector_store.py
│   ├── scripts/
│   │   ├── rebuild_index.py
│   │   └── rag_test.py
│   ├── agents/
│   │   ├── agent.py
│   │   └── tools.py
│   ├── api/
│   │   ├── main.py
│   │   └── routes/
│   │       ├── chat.py
│   │       └── transcribe.py
│   └── evaluation/
│       ├── evaluate.py
│       ├── cache_answers.py
│       └── qa_dataset.json
├── frontend/
│   ├── src/
│   │   ├── api-client.ts
│   │   └── chat-widget.ts
│   ├── vite.config.ts
│   └── vite.widget.config.ts
└── tests/
    ├── conftest.py
    ├── test_agent.py
    ├── test_api.py
    ├── test_evaluation.py
    ├── test_frontend.py
    └── test_pipeline_e2e.py
```

The README keeps the full technical documentation for readers who want to inspect the implementation in more detail.

6. Basic Commands

Only the main commands are needed to understand the project workflow and verification path.

```
# Build the vector index from transcript JSON files
uv run python backend/scripts/rebuild_index.py
```

```
# Run RAG Test from the terminal
uv run python backend/scripts/rag_test.py
```

```
# Run the test suite
uv run python -m pytest tests/ -v
```

```
# Run RAGAS evaluation
uv run python backend/evaluation/evaluate.py
```

```
# Start the API locally
uv run uvicorn backend.api.main:app \
  --reload --port 8000
```

```
# Build the embeddable widget
cd frontend && pnpm build:widget
```

7. Evaluation

The system was evaluated with RAGAS using a 5-question dataset. The evaluation measures faithfulness, answer relevancy, context precision, and context recall. Gemini 2.5 Flash was used as the judge LLM for this evaluation.

A cache layer (`cache_answers.py`) stores the agent's answers so the RAGAS metrics can be recalculated without repeated LLM calls during evaluation tuning.

The final scores were:

Metric	Score	Meaning
Faithfulness	0.96	The answer is supported by context
Answer relevancy	0.78	The answer responds to the question
Context precision	0.90	Retrieved chunks are relevant
Context recall	0.73	Expected context is covered

These results show that the system is strong at producing grounded answers. Context recall is the next improvement area, because better recall can help the system find more of the expected evidence.

8. Limitations and Next Steps

The evaluation scope is still limited. A larger set of questions would give a more complete view of the system, but the first 5 questions were useful as a realistic first step. They helped test if the agent could retrieve context, answer with sources, and expose the first areas for improvement.

With more time and with stronger approval from the owner of the information, the next step would be to test Cero inside the real website. This would require expanding the backend and integrating it more directly into the platform, so the widget can be tested with real users and real usage conditions.

The frontend was one of the biggest challenges of the project. The current version is functional, but it can continue improving after this first delivery.

Future improvements:

- Expand the evaluation dataset with more questions and more video types.
- Test the widget inside the real Plataforma Cero website after owner approval.
- Improve frontend interactions, animations, and mobile details.
- Add a feedback flow so users can report weak answers or missing context.

Conclusion

Migrant Archive Agent uses AI as a practical access layer for migrant narratives. The project takes long videos, turns them into searchable transcript chunks, stores them in a vector database, and exposes the result through Cero, an agent with tools, memory, API access, and an embeddable widget.

The technical decisions were shaped by the project requirements, the available tools, the need to work with Spanish and Catalan content, and the responsibility of testing the system locally before wider use.